```
Line 1 version: '3'
-
- services:
-   web:
5      image: robisenberg/myapp_web:prod
-      ports:
-        - "80:3000"
-      env_file:
-        - .env/production/database
10       - .env/production/web
-
-   redis:
-     image: redis
-
15  database:
-     image: postgres
-     env_file:
-       - .env/production/database
-     volumes:
20       - db_data:/var/lib/postgresql/data
-
- volumes:
-   db_data:
```

You'll notice a number of changes. For a start, we've completely removed the webpack_dev_server and selenium_chrome services: these were only needed for development and testing.

The remaining changes relate to the web service. We've removed the build attribute from the web service since building an image from a Dockerfile only works with Compose; with Swarm, we must specify a preexisting image to use. Here we specify the fully qualified name for the production image we pushed to Docker Hub: robisenberg/myapp_web:prod (line 5); in particular, we specify the version of this image tagged as prod.

We've also removed the web service's volumes attribute, which we previously used to mount our local Rails folder into the container (for live code reloading) and mount our gem cache (to speed up gem changes). Since we won't be developing or changing the code in production, neither of these are necessary.

We've changed the env_files to point to our production config (lines 9–10).

Finally, we've changed the mapped port for the web service so it exposes the Rails server on the default HTTP port 80 (line 7).